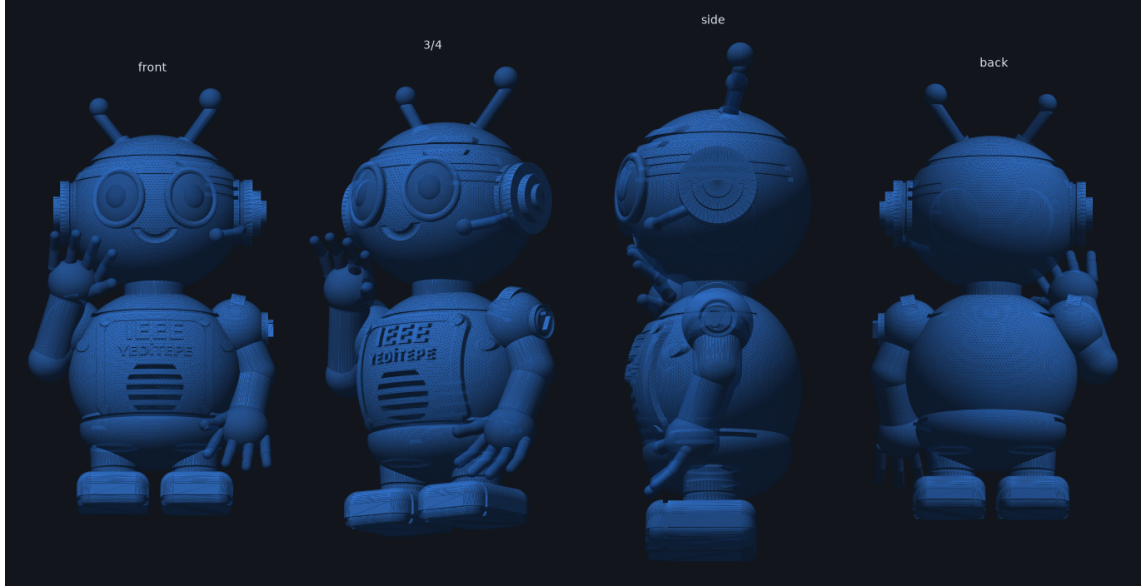


TEPECAN

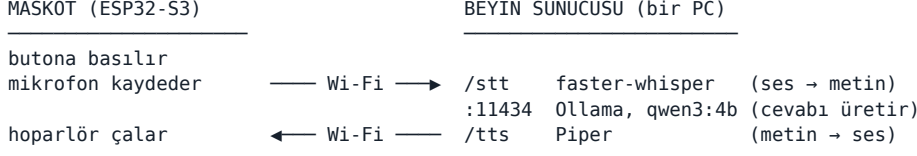
Konuřan maskot — yapım kılavuzu

3D baskı gövde · ESP32-S3 · yerel yapay zekâ · Türkçe sesli sohbet



1. Ne yapıyoruz

Maskot dinler, sorulanı yapay zekâyâ iletir, cevabı içindeki hoparlörden Türkçe olarak söyler. Ağır iş maskotun içinde değil, ağıdaki bir bilgisayarda çalışır. Maskotun içindeki kartın tek görevi mikrofonu okumak ve hoparlöre yazmak — bu sayede 512 MB RAM'li küçük bir kart yeter ve token maliyeti oluşmaz.



Cevap tamamlanmadan konuşmaya başlar: model ilk cümleyi bitirir bitirmez o cümle seslendirilir, kalanı arka planda üretilir. Algılanan gecikmeyi yarıya indiren tek değişiklik budur.

Verilen kararlar ve gerekçeleri

Karar	Neden
Beyin ayrı bir PC'de	Bu sınıf kartlarda 512 MB - 2 GB RAM var; en küçük dil modeli bile zor sığar, sığsa saniyede 1-3 token üretir. Bir cümle yarım dakika sürerdi.
Büyük kart alınmıyor	85 × 56 mm bir kart maskotun içine girmiyor: kapak açıklığının köşegeni 85.1 mm, o kartın köşegeni 85.8 mm. Kapalı PLA kasada da ısınıyor. Çözdüğü tek darboğaz zaten sunucuya taşındı.
Linux kartı yerine ESP32-S3	Raspberry Pi de, muadil aarch64 kartlar da Türkiye'de bulunamadı; ESP32-S3 her yerde var ve üçte bir fiyatta. Kart zaten ince istemci olduğu için işletim sistemine ihtiyaç yok: SD kart, açılış süresi ve sürücü avı tamamen kalkıyor.
Ses I2S üzerinden	ESP32-S3'ün dahili DAC'ı yok (klasik ESP32'de vardı, S3'te kaldırıldı), yani analog çıkış seçenek değil. MAX98357A hem DAC hem amfi olduğu için tek modülle çözülüyor.
Metin → ses de sunucuda	Piper'ın Türkçe medium sesi Zero 2 W'de gerçek zamandan yavaş kalabiliyor. Sunucuda çalıştırınca bu risk tamamen kalkar, Pi'ye hazır ses gelir.
Model: Qwen3 4B	Küçük modeller arasında çok dilli tarafı güçlü olanlardan; Türkçesi kullanılabilir. Gemma 3 4B ikinci aday, karşılaştırmaya değer.
Uyandırma kelimesi + buton	"Hey Tepecan" ile butonsuz tetikleme (openWakeWord, yerel). Metin→ses sunucuya taşındığı için Pi'de yer açıldı. Buton yedek olarak duruyor: gürültülü stantta uyandırmayı kapatıp butona dönebilirsin.
Ollama + faster-whisper + Piper	Üçü de ücretsiz ve yerel. Zincirin tamamı internetsiz çalışır, kullanım başına maliyet yoktur.

2. Malzeme listesi

Fiyatlar kabaca, sadece büyüklük fikri versin diye.

#	Parça	Neden bu	≈
1	ESP32-S3-DevKitC-1, N16R8	Maskotun beyni değil, kulağı ve ağzı: mikrofonu dinler, beyin sunucusuna gönderir, gelen cevabı çalar. N16R8 varyantı şart — 8 MB PSRAM'de kayıt tamponu ve Piper'dan dönen WAV duruyor. 64 × 28 mm, montaj deliği yok (tepsiye kelepçeyle bağlanıyor).	\$8
2	INMP441 I2S mikrofon modülü	Dijital çıkışlı, doğrudan karta bağlanıyor. L/R bacağı GND'ye gider, kod sol kanalı okuyor.	\$3
3	MAX98357A I2S amfi modülü	Hem DAC hem 3 W sınıf-D amfi. ESP32-S3'ün dahili DAC'ı olmadığı için sesin analoğa çevrilmesi burada oluyor. VIN 5 V ister, 3V3 değil. Hoparlör çıkışı vidalı klemens — lehim gerekmiyor.	\$4
4	Ø30 mm 4-8 Ω hoparlör	Gövdedeki yuva bu ölçüye göre: ızgara Ø30 mm, oturma omzu Ø32 mm, en fazla 8 mm gövde derinliği. Ø36 sığmıyor — göğüs küresel olduğu için o ayak izinde oturma omzu kartın önüne giriyor. 8 Ω olur, biraz daha kısık çalar.	\$3
5	5 V 2 A adaptör + USB-C kablo	Telefon şarjı iş görür. Namlu uçlu adaptör ALMA — kart USB-C ile besleniyor.	\$5
6	6 × 6 × 4.3 mm tactile switch + ince kablo	Sırt kapağındaki bas-konuş butonu. Uyandırma kelimesi varsayılan kapalı olduğu için şimdilik tek tetikleyici bu. Basılan kapağı (tepecan_buton.stl) sen basıyorsun.	\$1
7	M2.5 × 8 mm kendinden kılavuzlu vida (4 adet)	Tepsiyi gövde kulelerine sabitlemek için. Kılavuz delikleri Ø2.1 mm hazır.	\$2
8	M3 × 8 kendinden kılavuzlu vida (2 adet)	Sırt kapağını tutturmak için. Kasada yuvaları hazır.	\$1
9	Plastik kelepçe (2.5 mm, birkaç adet)	Kartı, amfiyi ve mikrofonu tepsiye bağlamak için. Tepside 3.6 mm'lik yuvalar hazır — kartta vida deliği olmadığı için montaj böyle.	\$1
10	Havya + lehim teli	Modüller lehimsiz geliyor. Toplam ~22 nokta: kartta tek header şeridi (bütün kullanılan pinler aynı kenarda), mikrofonda 6, amfide 5. Yarım saatlik iş.	—
11	PLA filament, mavi (~400 g)	Gövde. Güneşte/araçta kalacaksa PETG tercih et.	\$12
—	Beyin sunucusu: kulüpte zaten olan bir PC	16 GB RAM yeterli. 8-12 GB VRAM'li bir ekran kartı varsa cevaplar üç kat hızlanır. Kurulum: yazilim/sunucu/kur.sh	—

Kart alırken N16R8 yazısına dikkat: PSRAM'siz ESP32-S3 varyantları da satılıyor ve onlarda ses tamponları sığmıyor. Klasik ESP32 (S3 değil) da çalışır ama uyandırma kelimesi için S3'ün yapay zekâ komutları lazım. Kartta iki USB-C portu var; yükleme birinde olmazsa diğerini dene.

3. Gövde: 3D baskı

Basılacak dosyalar

Dosya	Ne	Adet
tepecan_body.stl	İçi boş gövde. Sırtta açıklık ve kapak oturma kenarı, iki vida boss'u, hoparlör adası, dört genel amaçlı kart kulesi, kablo yuvası.	1
tepecan_lid.stl	Sırt kapağı. Havalandırma yarıkları ve havşalı vida delikleriyle, baskı pozisyonunda.	1
tepecan_buton.stl	Kapaktaki butonun kapağı: Ø10 mm başlık, 5 mm sap. Yatık basılır, destek istemez.	1
tepecan_plaka.stl	Kart tepsisi: altı gövde kulelerine oturur, kart üstüne kelepçeyle bağlanır. 72 x 36 x 3 mm, ~20 dakika. Kart değişirse yalnız bu yeniden basılır.	1
tepecan_solid.stl	İçi dolu vitrin figürü. Elektronik koymayacaksan bunu bas, diğerlerini atla.	0-1

Baskı ayarları

Ayar	Değer	Not
Malzeme	PLA / PLA+	PETG daha ısıya dayanıklı
Katman	0.20 mm	Yüz için 0.16 daha temiz
Duvar	3 perimetre	
Dolgu	%10-15 gövde, %20 kapak	
Destek	Evet, ağaç/tree, her yerden	Çene altı, kollar, eller, kulaklık diskleri, mikrofon kolu, antenler
Brim	5 mm	Antenler için
Yönlendirme	Figür ayakta	Taban zaten düzleştirilmiş

Ölçüler: 240 mm boy (anten uçları dahil), 138 mm en, 94 mm derinlik. **Z eksenini en az 250 mm olan bir yazıcı gerekiyor** — Ender 3, P1S, X1C uygun; A1, A1 mini, MK4 gibi 180-210 mm'lik yazıcılara sığmaz. Kabaca 300-400 g filament ve 30-45 saat. Daha kısa baskı için modeli --height 180 ile yeniden üretebilirsin; cidar 3 mm ve vidalar M3 olarak kalır.

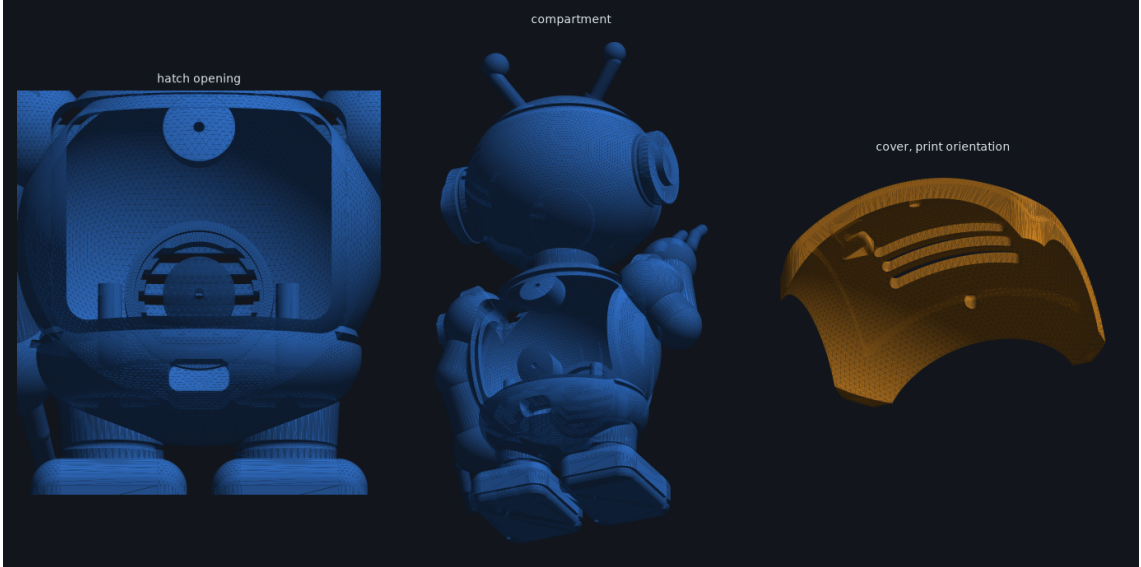
Gövdedeki elektronik detayları

Bunların hepsi modelde hazır; ölçüler govde/dogru.py ile her baskı öncesi ölçülüyor.

- **Göğüs ızgarası** — Ø30 mm dairesel alan içinde 5 yatay yarık, 2.7 mm yükseklik, merkezi tabandan 65 mm'de. YEDİTEPE yazısına 3.2 mm pay var.
- **Hoparlör yuvası** — ızgaranın arkasında düz omuzlu bir ada: göğüs duvarı küresel olduğu için düz yüzü bir hoparlör ona yaslanamıyor, ada sarkmayı dolduruyor. Cep Ø32 mm, omuz 2.4 mm. Ø40 mm hoparlör bu yüzden sığmıyor: o ayak izinde omuz y = 11.9 mm'ye kadar geriliyor ve kartın önüne giriyor.
- **Kart montaj kuleleri** — Ø6 mm, dört adet, **karttan bağımsız** 44 x 28 mm aralıkta. Üst yüzeyleri tabandan 63.8 mm'de, tepelerinde M2.5 için Ø2.1 mm kılavuz delik, 8 mm derin. Kartın kendi delik deseni bu kulelerde değil, üstlerine vidalanan tepside.

- **Adaptör plakası** — 66 × 36 × 6 mm ayrı parça. Gövdeyi kart seçiminden kurtarıyor: kart değişirse 30-45 saatlik gövde değil, 20 dakikalık plaka yeniden basılıyor. ESP32-S3-DevKitC-1'de montaj deliği olmadığı için kart vidalanmıyor, kelepçeyle bağlanıyor; köşe yuvalarına da amfi ve mikrofon bağlanıyor.
- **Kapak açıklığı** — dıştan 73 × 62 mm, oturma kenarından geçiş 65 × 54 mm, köşegen 85 mm. Genişliği kart değil, kartı vidalayan tornavida belirledi: plakadaki vidalar $x = \pm 29$ mm'de ve açıklığın alt köşesine yakın; bu ölçüde kenara 3.7 mm kalıyor, tornavida rahat giriyor. Plaka açıklıktan yan yatırılarak geçiyor (köşegen 75 < 85 mm).
- **Vida boss'ları** — Ø18.7 mm, kavite yüzeyinden 12 mm içeri; M3 vidaya kapak eti ile birlikte 15 mm dış kalıyor.
- **Buton yuvası** — sırt kapağının iç yüzünde: 6.6 mm kare cep, 4.6 mm derin, önünde 2 mm sap kılavuzu, dışa Ø4.2 mm delik.
- **Kablo yuvası** — alt sırtta 15.6 × 7.8 mm, tabandan 38 mm'de; micro-USB fişi kıvrılmadan geçiyor.

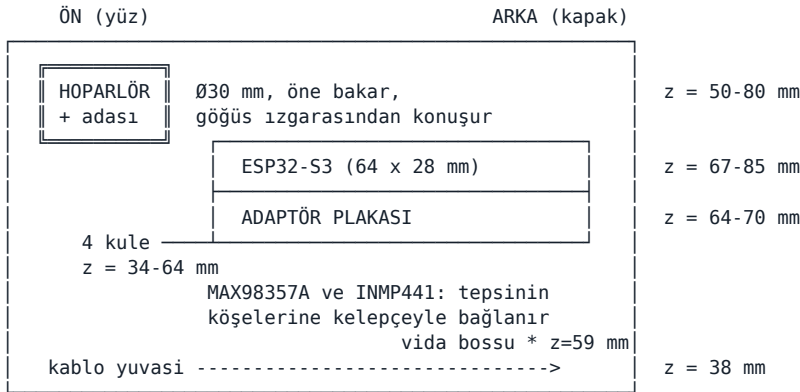
Işıklı gözler denenmedi: RGB LED'leri boyundan geçen bir ışık kanalıyla gözlere taşımak, gözleri şeffaf filamentten ayrı basmayı ve kafayı iki parça yapmayı gerektiriyor. Maskotu en canlı gösterecek şey bu, ama yapımı belirgin zorlaştırdığı için modele girmedi.



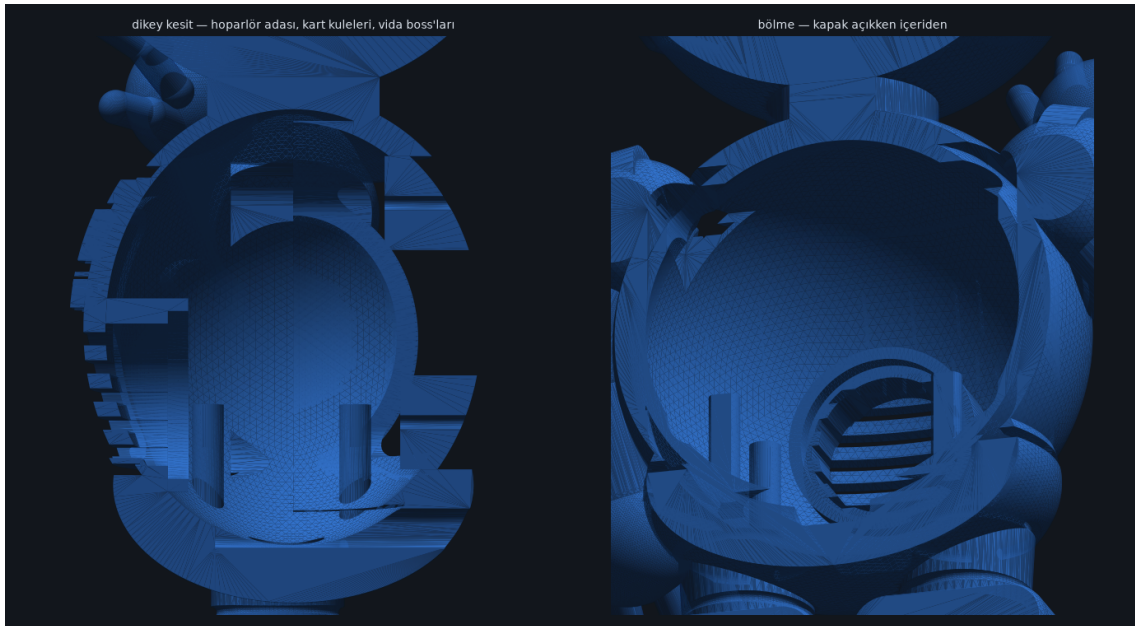
Soldan: sırt açıklığı ve kapak oturma kenarı; bölmenin dıştan görünüşü; kapak, baskı pozisyonunda (dış yüz yukarı, destek gerekmez).

4. İç yerleşim ve montaj

İç boşluk elipsoid olduğu için kutu ölçüsünden cömert: en geniş yerinde 90 × 75 mm, toplam yükseklik 87 mm. Boşluk tabandan 34 mm'de başlıyor, 121 mm'de bitiyor. Aşağıdaki yükseklikler tabandan ölçülü.



Kart, alt vida bossunun (z = 49.8-68.5 mm) tepesinin üstünde duruyor; bu yüzden arka sınırı boss değil kavite duvarı belirliyor. Sığan en büyük kart: 30 mm derinse 74.5 mm, 40 mm derinse 64.8 mm, 50-55 mm derinse 63.4 mm genişlik (derin kartlar plakada 7-10 mm geriye kaydırılıyor). Plakanın üstünde 40 mm yükseklik var. `dogrula.py` bu sayıları her baskı öncesi ölçüyor.



Solda gövdenin dikey kesiti: kart kuleleri ve hoparlör adası. Sağda kapak açırken bölmenin içi.

Montaj sırası

Gövde baskısı karta bağlı değil: plakayı kart gelmeden de basabilirsin, ama delik desenini kartın kendisinden kumpasla ölçüp üretmek en güvenlisi.

- Gövdeyi ve kapağı bas, destekleri temizle. Kapağın açıklığa boşlukla oturduğunu kuru kuruya dene.
- Kart elindeyken montaj deliklerinin aralığını kumpasla ölç. `tepecan_model.py` içinde `BOARD_HOLE_X` / `BOARD_HOLE_Y` değerlerini gir (yarı ölçü: 58 × 23 mm delik için 58/2 ve 23/2), yeniden üret, `tepecan_plaka.stl`'i bas. Kart 35 mm'den derinse `BOARD_OFFSET_Y` ile geriye kaydır: 50 mm için -7, 55 mm için -10 mm.

- Hoparlörü göğüs izgarasının arkasındaki adaya, düz omza yüzü öne bakacak şekilde otur ve kenarından yapıştır. **Arkasını olabildiğince kapalı tut** — omzun çevresini silikonla sızdırmaz yapmak ses kalitesini gözle görülür artırır, bedava kazançtır.
- İşlemcinin üstüne soğutucuyu yapıştır (kartı plakaya vidalamadan önce yap, sonra elin girmez).
- Kartı tepsiye kelepçeyle bağla (kartta vida deliği yok; tepside kartın altından geçen iki sıra yuva var). MAX98357A ve INMP441'i köşe yuvalarına bağla, kabloları kısa kes. Bu işi gövdenin dışında yap — tepsi avucunda dururken çok rahat.
- Hazır plakayı açıklıktan **yan yatırarak** içeri sok (düz geçmez, köşegen 75 mm) ve dört M2.5 vidayla gövde kulelerine sabitle. Kartın güç girişi kablo yuvasına baksın.
- Hoparlörün iki telini MAX98357A'nın vidalı klemensine sıkıştır.
- Buton kapağını (tepecan_buton.stl) sırt kapağının **dışından** deliğe geçir. Switch'i kapağın iç yüzündeki cebe, pistonu kapağa bakacak şekilde bastır; sap tam pistonu dayanır. İki telini kartın bir GPIO hattına ve GND'ye bağla. Telleri kapağı kapatırken sıkışmayacak kadar uzun bırak.
- Güç kablosunu alt sırttaki yuvadan geçir, karta tak.
- Kapağı iki M3 vidayla kapat. Vidalar havşalı deliklere oturur, başları yüzeyle aynı hizada kalır.

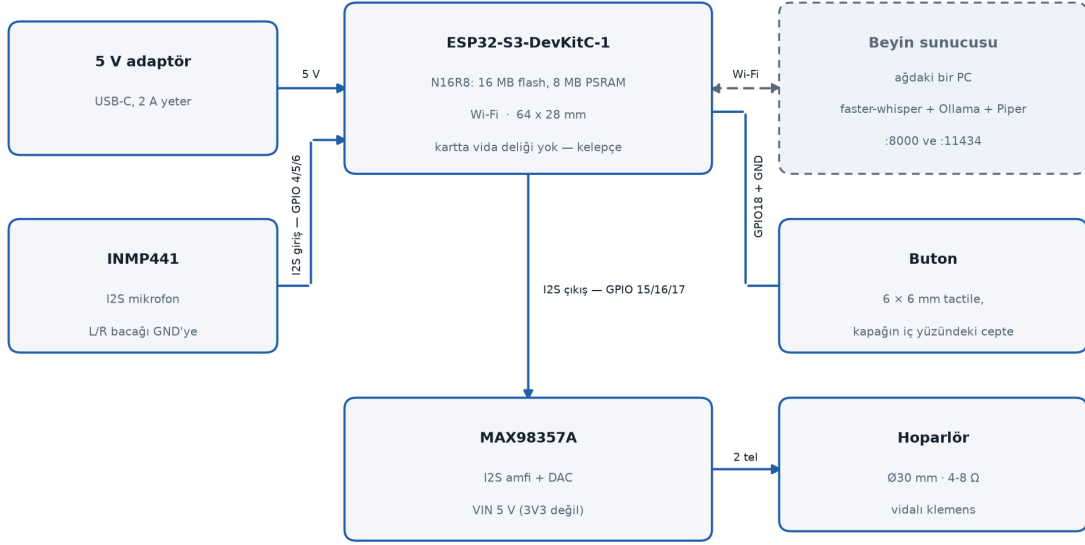
Kablolama

Nereden	Nereye	Not
INMP441 VDD / GND	Kartın 3V3 ve GND pini	L/R bacağı da GND'ye
INMP441 SCK / WS / SD	GPIO 4 / 5 / 6	I2S giriş
MAX98357A VIN / GND	Kartın 5Vin ve GND pini	5 V ister, 3V3 değil
MAX98357A BCLK / LRC / DIN	GPIO 15 / 16 / 17	I2S çıkış
MAX98357A çıkışı	Hoparlör (+/-)	Vidalı klemens, kutup önemsiz
Buton bacak 1	Bir GPIO hattı	Kodda TEPECAN_BUTTON_PIN / TEPECAN_BUTTON_CHIP
Buton bacak 2	GND	
5 V adaptör	Kartın güç girişi	Varsa veri portuna değil, güç portuna

Kullanılan pinlerin hepsi kartın tek kenarında: 5Vin, GND, 18, 17, 16, 15, 6, 5, 4, 3V3. Yani tek bir header şeridi lehimlemek yetiyor, diğer kenara hiç dokunma.

Tepecan — bağlantı şeması

Maskotun içindeki tüm kablolama. Beyin sunucusu ağ üzerinden bağlanır, kablosu yoktur.



Notlar

1. ESP32-S3'ün dahili DAC'ı yok (klasik ESP32'de vardı), o yüzden ses tamamen I2S: MAX98357A hem DAC hem amfi.
2. Bütün pinler kartın tek kenarında: 5Vin, GND, 18, 17, 16, 15, 6, 5, 4, 3V3. Tek header şeridi lehimlemek yetiyor.
3. INMP441 3V3'ten beslenir, MAX98357A 5V'tan.
4. Butonun kutbu yok, iki bacağı GPIO18 ve GND'ye gider.
5. Uyandırma kelimesi verseyilan kapalı; buton tetikler. Açmak için bkz. yazılım/esp32/README.md.

Bağlantı şeması.

Isı: kapalı PLA kasada bu sınıf bir kart boşta 50-55 °C civarında kalır. Kapaktaki üç yarık ve kablo yuvası hafif hava akışı sağlıyor, soğutucuyla birlikte sorun çıkarmaz. PLA 60 °C dolayında yumuşamaya başlar; maskot güneşte veya araçta kalacaksa gövdeyi PETG bas.

5. Beyin sunucusu kurulumu

Kulüpteki herhangi bir PC. Linux varsayılıyor; Windows'ta WSL2 içinde de çalışır.

Ollama ve model

```
curl -fsSL https://ollama.com/install.sh | sh

ollama pull qwen3:4b          # ana aday
ollama pull gemma3:4b        # ikinci aday, karşılaştırmak için

# Ağdan erişilebilir olsun diye:
OLLAMA_HOST=0.0.0.0 ollama serve
```

Piper (Türkçe ses)

```
mkdir -p ~/piper && cd ~/piper
# piper çalıştırılabilir dosyasını ve şu ikisini buraya indir:
#   tr_TR-dfki-medium.onnx
#   tr_TR-dfki-medium.onnx.json

echo "Merhaba, ben Tepecan." | ./piper --model tr_TR-dfki-medium.onnx --output_file test.wav
aplay test.wav                # sesi duyuyorsan Piper hazır
```

Ses servisi

```
pip install -r requirements-sunucu.txt
uvicorn beyin:app --host 0.0.0.0 --port 8000

# GPU varsa belirgin daha doğru:
STT_DEVICE=cuda STT_MODEL=medium uvicorn beyin:app --host 0.0.0.0 --port 8000
```

Sunucunun ağdaki adını tepecan-beyin yaparsan Pi tarafında hiçbir ayar değiştirmen gerekmez. Yapamıyorsan Pi'de TEPECAN_BRAIN=http://192.168.1.42 gibi ver.

Kontrol: başka bir makineden curl http://tepecan-beyin.local:8000/health komutu {"ok": true, ...} döndürmeli.

6. Maskot (Pi) kurulumu

Raspberry Pi OS Lite, 64 bit. Kurulum sırasında Wi-Fi ve SSH'ı aç.

Firmware'i yükle

ESP-IDF v5.1 veya üstü gerekiyor. En kolay yol VS Code + Espressif'in ESP-IDF eklentisi; eklenti toolchain'i kendisi kuruyor.

```
cd yazilim/esp32
idf.py set-target esp32s3
idf.py build
idf.py -p COM3 flash monitor          # Linux'ta /dev/ttyACM0 gibi
```

Seri monitörde Tepecan başlıyor ve ardından kayıtlı ağ yok - kurulum noktası açılıyor görmelisin.

Ağa bağla — bilgisayara gerek yok

Wi-Fi bilgisi koda gömülü değil, NVS'e yazılıyor. Kart kayıtlı ağ bulamazsa kendi erişim noktasını açıyor:

- Telefondan **Tepecan-Kurulum** ağına bağlan (şifresiz).
- Tarayıcıda <http://192.168.4.1> aç.
- Wi-Fi adı, şifresi ve beyin sunucusunun adresini gir (<http://sunucunun-IP-adresi:8000>).
- Kaydet — kart yeniden başlayıp o ağa bağlanıyor.

Kulüpten etkinliğe geçerken ağ değiştirmek için de aynı portal yeter; USB yalnız ilk yüklemede lazım. Sonrasında prize takılı durduğu sürece çalışır.

Uyandırma kelimesi varsayılan KAPALI geliyor: çalışması için ifadeye özel eğitilmiş bir microWakeWord modeli gerekiyor ve o model depoda yok. Şimdilik sırt kapağındaki buton tetikliyor. Açma yolu ve model eğitimi [yazilim/esp32/README.md](#) içinde.

7. Ayarlar

Maskot tarafındakiler yazılım/esp32/main/tepecan.h içinde derleme sabitleri, sunucu tarafındakiler ortam değişkeni.

Ayar	Varsayılan	Ne işe yarar
PIN_MIC_SCK / WS / SD	4 / 5 / 6	Mikrofonun I2S pinleri
PIN_SPK_BCLK / LRC / DIN	15 / 16 / 17	Amfinin I2S pinleri
PIN_BUTON	18	Butonun bağlı olduğu GPIO
SESSİZLİK_MS	800	Bu kadar sessizlikten sonra kayıt biter
EN_UZUN_KAYIT_SN	10	Kayıt tavanı
UYANDIRMA_ETKİN	0	1 yapınca uyandırma kelimesi derlenir
WAKE_ESİK	0.85	Uyandırma güven eşiği (yükselt = az yanlış tetik)
BEYİN_VARSAYILAN	tepecan-beyin.local:8000	Kurulum sayfasından değişir
TEPECAN_MODEL	qwen3:4b	Sunucuda: Ollama modeli
STT_MODEL	small	Sunucuda: whisper boyutu (small / medium)
STT_DEVICE	cpu	Sunucuda: cpu veya cuda
PIPER_MODEL	~/piper/tr_TR-dfki-medium.onnx	Sunucuda: Türkçe ses

Mikrofon sesi kısık ya da bozuk gelirse yazılım/esp32/main/ses.c içindeki kazanç kaydırmasına (>> 11) ve MIC_KANAL_SOL ayarına bak; donanımda denenmemiş iki nokta bunlar.

8. Küçük modelden iyi cevap almanın yolu

Model 4 milyar parametrelidir. Ondan bilgiyi *bilmesini* değil, verdiğimiz metni *okumasını* istiyoruz. Kaliteyi belirleyen şey model değil, aşağıdakiler:

- **bilgiler.txt dosyasını doldur.** Kulübe dair her gerçek buraya. Her soruda modele veriliyor. Doğru cevabı sağlayan asıl mekanizma bu — en yüksek getirili iş, modeli değiştirmek değil bu dosyayı yazmaktır.
- **2-3 cümle sınırını gevşetme.** persona.txt bunu dayatıyor. Küçük modeller uzadıkça dağılır, üstelik hoparlörden uzun cevap dinlemek sıkıcıdır.
- **Hafızayı artırma.** Altı mesaj tutuluyor, üç dakika sessizlikte sıfırlanıyor. Uzun geçmiş küçük modelde kaliteyi düşürür.
- **repeat_penalty 1.15** kendini tekrarlama döngüsünü kesiyor, **num_predict 160** cevabı kısa tutuyor.
- Qwen3 düşünme modunda <think> etiketi üretebiliyor; hem kapatılıyor hem de metinden temizleniyor. Yoksa hoparlör modelin iç monologunu okur.
- **Model seçimini ölçerek yap:** aynı on soruyu qwen3:4b ve gemma3:4b'ye sor, kulüpten birkaç kişiyle kör karşılaştır. Bir akşamlık iş, kararı bitirir.

9. Test ve sorun giderme

Belirti	Muhtemel sebep	Ne yapmalı
Hiç ses kaydedilmiyor	Yanlış ses cihazı seçili	arecord -l ve python3 -m sounddevice ile USB kartını bul, TEPECAN_MIC'e yaz
Buton çalışmıyor	gpiozero yalnız Raspberry Pi'de çalışır	python-periphery kurulu mu bak; TEPECAN_BUTTON_CHIP ve TEPECAN_BUTTON_PIN'i kartının pin haritasına göre ayarla
Kayıt hemen bitiyor	VAD gürültüyü konuşma sanıyor ya da mikrofon sessiz	alsamixer ile giriş seviyesini yükselt; kodda Vad(2) yerine Vad(1) dene
Kayıt hiç bitmiyor	Ortam gürültülü, sessizlik algılanmıyor	Vad(3) yap; SILENCE_MS değerini 600'e düşür
STT hatası / bağlanamıyor	Sunucu adresi yanlış veya güvenlik duvarı	karttan curl http://tepecan-beyin.local:8000/health dene
Cevap geliyor ama ses yok	Piper modeli sunucuda bulunamıyor	Sunucu günlüğüne bak; PIPER_MODEL yolunu tam ver
Ses cızırtılı, kesik	Adaptör zayıf	5 V 3 A adaptör kullan; powerbank'ten besleme
Model saçmalıyor, konuyu kaçırıyor	bilgiler.txt boş ya da çok uzun	Doldur ama 1500 karakteri geçme
Cevaplar çok yavaş	Sunucuda GPU yok	Daha küçük model (qwen3:1.7b) veya STT_MODEL=base dene
Hoparlörden model düşüncesi okunuyor	<think> etiketi sızmış	Ollama sürümünü güncelle; think:false destekli olmalı
Durmadan kendi kendine uyanıyor	Uyandırma eşiği düşük, ortam gürültülü	TEPECAN_WAKE_THRESHOLD=0.6, sonra TEPECAN_WAKE_HITS=3
Uyandırma kelimesine hiç tepki vermiyor	Eşik yüksek, mikrofon kısık ya da model yolu yanlış	Açılıştaki "Uyandırma kelimesi hazır" satırını gör; eşiği 0.4 yap

Beklenen gecikme

Sunucu	Soru sonundan ilk sese
Ekran kartlı PC (8-12 GB VRAM)	yaklaşık 2-3 saniye
Ekran kartsız PC, qwen3:4b	yaklaşık 5-7 saniye
Ekran kartsız PC, qwen3:1.7b	yaklaşık 3-4 saniye

10. Kodlar

yazilim/sunucu/beyin.py — beyin sunucusu (ses↔metin)

```
#!/usr/bin/env python3
"""
Tepecan beyin sunucusu – ses->metin ve metin->ses.

Bu işler maskotun içindeki ESP32 için fazla ağır; buraya alındı. LLM ayrı
bir servis (Ollama) olarak aynı makinede çalışır, sunucu ona bağlanır.

uvicorn beyin:app --host 0.0.0.0 --port 8000

Ortam değişkenleri:
    STT_MODEL=small          CPU'da makul, Türkçesi iyi (varsayılan)
    STT_DEVICE=cuda          GPU varsa
    PIPER_MODEL=/yol/tr_TR-dfki-medium.onnx
"""

import os
import pathlib
import subprocess
import tempfile

import requests
from fastapi import FastAPI, File, Request, UploadFile
from fastapi.responses import Response
from pydantic import BaseModel
from faster_whisper import WhisperModel

STT_MODEL = os.getenv("STT_MODEL", "small")
STT_DEVICE = os.getenv("STT_DEVICE", "cpu")
STT_COMPUTE = os.getenv("STT_COMPUTE", "int8" if STT_DEVICE == "cpu" else "float16")

OLLAMA_URL = os.getenv("OLLAMA_URL", "http://127.0.0.1:11434")
OLLAMA_MODEL = os.getenv("TEPECAN_MODEL", "qwen3:4b")

PIPER_BIN = os.getenv("PIPER_BIN", "piper")
PIPER_MODEL = os.getenv("PIPER_MODEL",
                        os.path.expanduser("~/piper/tr_TR-dfki-medium.onnx"))

BURASI = pathlib.Path(__file__).resolve().parent

def _oku(ad, yedek=""):
    try:
        return (BURASI / ad).read_text(encoding="utf-8").strip()
    except OSError:
        return yedek

PERSONA = _oku("persona.txt", "Sen Tepecan'sın, Yeditepe IEEE kulübünün maskotu.")
BILGILER = _oku("bilgiler.txt")

print(f"faster-whisper yükleniyor: {STT_MODEL} ({STT_DEVICE}/{STT_COMPUTE})")
whisper = WhisperModel(STT_MODEL, device=STT_DEVICE, compute_type=STT_COMPUTE)
print("Hazır.")

app = FastAPI(title="Tepecan beyin")

@app.get("/health")
def health():
    return {"ok": True, "stt": STT_MODEL, "device": STT_DEVICE}

def _cozumle(wav_baytlari: bytes) -> str:
    with tempfile.NamedTemporaryFile(suffix=".wav") as tmp:
        tmp.write(wav_baytlari)
        tmp.flush()
        segments, info = whisper.transcribe(
            tmp.name,
            language="tr",          # dili sabitlemek tahmin adımını atlatır, hızlandırır
            vad_filter=True,
            beam_size=1,           # sohbet için hız > son kırıntı doğruluk
        )
```

```

    )
    text = " ".join(s.text.strip() for s in segments).strip()

    print(f"STT [{info.duration:.1f} sn]: {text}")
    return text

@app.post("/stt")
async def stt(audio: UploadFile = File(...)):
    """WAV dosya yüklemesi al, Türkçe metin döndür (Python istemcisi)."""
    return {"text": _cozumle(await audio.read())}

@app.post("/stt-raw")
async def stt_raw(request: Request):
    """Ham WAV gövdesi al, Türkçe metin döndür.

    ESP32 için: mikrodenetleyicide multipart/form-data kurmak gereksiz
    zahmet, gövdeye doğrudan WAV koyup göndermek yetiyor.
    """
    return {"text": _cozumle(await request.body())}

class SohbetIstek(BaseModel):
    text: str

@app.post("/chat")
def chat(istek: SohbetIstek):
    """Soruyu Ollama'ya sor, cevabı düz metin olarak döndür.

    Akan JSON'u ayrıştırmayı ve cümlelere bölmeyi burada yapıyoruz;
    ESP32'ye yaptırmak gereksiz karmaşıklık. Kart yalnız soruyu yollayıp
    cevabı alıyor.
    """
    mesajlar = [{"role": "system", "content": PERSONA + "\n\n" + BILGILER},
                 {"role": "user", "content": istek.text}]
    r = requests.post(
        f"{OLLAMA_URL}/api/chat",
        json={"model": OLLAMA_MODEL, "messages": mesajlar, "stream": False,
              "options": {"temperature": 0.7, "num_predict": 200}},
        timeout=120,
    )
    r.raise_for_status()
    cevap = r.json().get("message", {}).get("content", "").strip()

    # qwen3 gibi modeller düşünme bloğu yazabiliyor; seslendirmeden önce at.
    if "</think>" in cevap:
        cevap = cevap.split("</think>", 1)[1].strip()

    print(f"LLM: {cevap[:80]}")
    return {"text": cevap}

class TTSIstek(BaseModel):
    text: str

@app.post("/tts")
def tts(istek: TTSIstek):
    """Metin al, WAV döndür. Piper burada çalışır, Pi'de değil."""
    with tempfile.NamedTemporaryFile(suffix=".wav") as tmp:
        subprocess.run(
            [PIPER_BIN, "--model", PIPER_MODEL, "--output_file", tmp.name],
            input=istek.text.encode("utf-8"),
            stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
            check=True,
        )
        tmp.seek(0)
        wav = open(tmp.name, "rb").read()

    print(f"TTS [{len(wav)/1024:.0f} KB]: {istek.text[:60]}")
    return Response(content=wav, media_type="audio/wav")

```

yazilim/esp32/main/tepecan.h — ayarlar ve pinler

```
/* Tepecan - ESP32-S3 istemcisi, ortak ayarlar.
 *
 * Kart: ESP32-S3-DevKitC-1 N16R8 (16 MB flash, 8 MB PSRAM).
 * Ses: INMP441 (I2S mikrofon) + MAX98357A (I2S amfi).
 *
 * Ağır iş burada değil: ses-metin, dil modeli ve metin-ses ağıdaki beyin
 * sunucusunda (yazilim/sunucu/beyin.py) çalışıyor. Bu kartın işi mikrofonu
 * okumak, uyandırma kelimesini yakalamak ve hoparlöre yazmak.
 */
#pragma once

#include <stdbool.h>
#include <stdint.h>

/* ----- pinler */
/* TFT yok, bu yüzden S3'ün hemen bütün GPIO'ları serbest. Aşağıdakiler
 * DevKitC-1'de header'a çıkan, açılışta özel anlamı olmayan hatlar. */
#define PIN_MIC_SCK 4 /* INMP441 SCK (bit clock) */
#define PIN_MIC_WS 5 /* INMP441 WS (word select) */
#define PIN_MIC_SD 6 /* INMP441 SD (veri çıkışı) */

#define PIN_SPK_BCLK 15 /* MAX98357A BCLK */
#define PIN_SPK_LRC 16 /* MAX98357A LRC */
#define PIN_SPK_DIN 17 /* MAX98357A DIN */

#define PIN_BUTON 18 /* kapaktaki tactile switch, diğer bacak GND */

/* INMP441'in L/R bacağı GND'ye bağlıysa sol kanaldan konuşur. */
#define MIC_KANAL_SOL true

/* ----- ses */
#define ORNEK_HZ 16000 /* STT, VAD ve uyandırma modeli 16 kHz ister */
#define BLOK_ORNEK 320 /* 20 ms - webrtcvad ve mWW ile uyumlu */
#define EN_UZUN_KAYIT_SN 10 /* bundan sonrası kesiliyor */
#define SESSIZLIK_MS 800 /* bu kadar sessizlikten sonra kayıt biter */

/* ----- uyandırma */
/* Uyandırma kelimesi varsayılan olarak KAPALI. Açmak için:
 * 1. Burayı 1 yap.
 * 2. main/idf_component.yml içindeki üç bağımlılığın yorumunu kaldır.
 * 3. Eğitilmiş modeli "model" bölmesine yükle (bkz. README).
 *
 * Kapalı geliyor çünkü model dosyası olmadan zaten çalışmıyor, ve açıkken
 * derleme kayıt sunucusundan üç bileşen indirmeye çalışıyor. Kapalıyken
 * proje kutudan çıktığı gibi derleniyor; buton tetikleyici olarak yeterli.
 *
 * DİKKAT: uyandırma.cc donanımda denenmedi. microWakeWord'ün beklediği
 * öznitelik çıkarıcı (eski "microfrontend" API'si) güncel esp-tflite-micro
 * sürümlerinde yok, ayrı bir bileşen olarak geliyor - açıldığında derleme
 * hatası alırsan ilk bakılacak yer orası.
 */
#define UYANDIRMA_ETKIN 0

/* microWakeWord modeli SPIFFS'te. Dosya yoksa program çalışır, yalnız
 * buton tetikler - uyandırma kelimesi sessizce devre dışı kalır. */
#define WAKE_MODEL_YOLU "/model/hey_tepecan.tflite"
#define WAKE_ESIK 0.85f /* 0.1, yükseltmek yanlış tetiklemeyi azaltır */
#define WAKE_ARDISIK 2 /* üst üste kaç pencere eşiği geçmeli */

/* ----- ağ */
#define BEYIN_VARSAYILAN "http://tepecan-beyin.local:8000"
#define HTTP_ZAMAN_ASIMI_MS 60000

/* ----- durum */
typedef enum {
    DURUM_BEKLIYOR, /* uyandırma kelimesi ya da buton bekleniyor */
    DURUM_DINLIYOR, /* kayıt sürüyor */
    DURUM_DUSUNUYOR, /* sunucu cevap üretiyor */
    DURUM_KONUSUYOR, /* hoparlör çalışıyor */
    DURUM_HATA,
} tepecan_durum_t;

void durum_yaz(tepecan_durum_t d);
const char *durum_adi(tepecan_durum_t d);
```

yazilim/esp32/main/main.c — akış

```
/* Tepecan - ESP32-S3 istemcisi.
 *
 * uyandırma kelimesi ya da buton
 * -> kayıt (sen susunca biter)
 * -> /stt-raw (ses -> metin)
 * -> /chat (metin -> cevap)
 * -> /tts (cevap -> ses)
 * -> hoparlör
 *
 * Bütün ağır iş beyin sunucusunda. Burada yalnız akış ve zamanlama var.
 */
#include "tepecan.h"
#include "ses.h"
#include "ag.h"
#include "uyandırma.h"

#include <string.h>
#include <stdlib.h>
#include "driver/gpio.h"
#include "esp_log.h"
#include "esp_heap_caps.h"
#include "esp_spiffs.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

static const char *ETIKET = "tepecan";
static tepecan_durum_t g_durum = DURUM_BEKLIYOR;

const char *durum_adi(tepecan_durum_t d)
{
    switch (d) {
        case DURUM_BEKLIYOR: return "bekliyor";
        case DURUM_DINLIYOR: return "dinliyor";
        case DURUM_DUSUNUYOR: return "düşünüyor";
        case DURUM_KONUSUYOR: return "konuşuyor";
        default: return "hata";
    }
}

void durum_yaz(tepecan_durum_t d)
{
    if (d == g_durum) return;
    g_durum = d;
    ESP_LOGI(ETIKET, "durum: %s", durum_adi(d));
}

/* ----- WAV */
/* 44 baytlık standart başlık; whisper tarafında sorunsuz açılıyor. */
static void wav_basligi(uint8_t *b, uint32_t veri_boy, uint32_t hz)
{
    uint32_t bayt_hz = hz * 2;
    memcpy(b, "RIFF", 4);
    uint32_t riff = 36 + veri_boy; memcpy(b + 4, &riff, 4);
    memcpy(b + 8, "WAVEfmt ", 8);
    uint32_t fmt = 16; memcpy(b + 16, &fmt, 4);
    uint16_t pcm = 1, kanal = 1; memcpy(b + 20, &pcm, 2); memcpy(b + 22, &kanal, 2);
    memcpy(b + 24, &hz, 4);
    memcpy(b + 28, &bayt_hz, 4);
    uint16_t hiza = 2, bit = 16; memcpy(b + 32, &hiza, 2); memcpy(b + 34, &bit, 2);
    memcpy(b + 36, "data", 4); memcpy(b + 40, &veri_boy, 4);
}

/* Gelen WAV'ın veri bölümünü ve örnekleme hızını bul. */
static const int16_t *wav_coz(const uint8_t *wav, size_t boy,
                              size_t *ornek, uint32_t *hz)
{
    if (boy < 44 || memcmp(wav, "RIFF", 4) != 0) return NULL;
    memcpy(hz, wav + 24, 4);
    size_t p = 12;
    while (p + 8 <= boy) {
        uint32_t parca_boy;
        memcpy(&parca_boy, wav + p + 4, 4);
        if (memcmp(wav + p, "data", 4) == 0) {
            if (p + 8 + parca_boy > boy) parca_boy = boy - p - 8;

```

```

        *ornek = parca_boy / 2;
        return (const int16_t *) (wav + p + 8);
    }
    p += 8 + parca_boy + (parca_boy & 1);
}
return NULL;
}

/* ----- buton */
static void buton_kur(void)
{
    gpio_config_t c = {
        .pin_bit_mask = 1ULL << PIN_BUTTON,
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE,
    };
    gpio_config(&c);
}

static bool buton_basili(void) { return gpio_get_level(PIN_BUTTON) == 0; }

/* ----- akış */
/* Tetik gelene kadar dinler. Dönen değer: "buton" ya da "kelime". */
static const char *tetik_bekle(void)
{
    static int16_t blok[BLOK_ORNEK];
    uyandirma_sifirla();
    while (true) {
        if (mik_oku(blok, BLOK_ORNEK) != ESP_OK) continue;

        if (buton_basili()) {
            while (buton_basili()) vTaskDelay(pdMS_TO_TICKS(20));
            return "buton";
        }
        if (uyandirma_dinle(blok, BLOK_ORNEK)) return "kelime";
    }
}

/* Sen susana kadar kaydeder. WAV'ı PSRAM'de döndürür. */
static uint8_t *sessizlige_kadar_kaydet(size_t *wav_boy)
{
    const size_t en_cok = (size_t)ORNEK_HZ * EN_UZUN_KAYIT_SN;
    uint8_t *wav = heap_caps_malloc(44 + en_cok * 2, MALLOC_CAP_SPIRAM);
    if (!wav) { ESP_LOGE(ETIKET, "kayıt için PSRAM yetmedi"); return NULL; }

    int16_t *veri = (int16_t *) (wav + 44);
    size_t yazilan = 0;
    int sessiz_blok = 0;
    const int sessiz_sinir = SESSIZLIK_MS / (BLOK_ORNEK * 1000 / ORNEK_HZ);
    bool konusma_basladi = false;

    while (yazilan + BLOK_ORNEK <= en_cok) {
        if (mik_oku(veri + yazilan, BLOK_ORNEK) != ESP_OK) break;
        bool ses = ses_var_mi(veri + yazilan, BLOK_ORNEK);
        yazilan += BLOK_ORNEK;

        if (ses) { konusma_basladi = true; sessiz_blok = 0; }
        else if (konusma_basladi && ++sessiz_blok >= sessiz_sinir) break;
        /* Hiç konuşma başlamadıysa 3 saniye sonra vazgeç. */
        else if (!konusma_basladi && yazilan > (size_t)ORNEK_HZ * 3) {
            free(wav);
            return NULL;
        }
    }
    if (!konusma_basladi) { free(wav); return NULL; }

    wav_basligi(wav, yazilan * 2, ORNEK_HZ);
    *wav_boy = 44 + yazilan * 2;
    ESP_LOGI(ETIKET, "kayıt bitti: %.1f sn", (float)yazilan / ORNEK_HZ);
    return wav;
}

static void konus(const char *metin)
{

```

```

size_t wav_boy = 0;
uint8_t *wav = beyin_tts(metin, &wav_boy);
if (!wav) return;

size_t ornek = 0;
uint32_t hz = ORNEK_HZ;
const int16_t *pcm = wav_coz(wav, wav_boy, &ornek, &hz);
if (pcm) {
    durum_yaz(DURUM_KONUSUYOR);
    hoparlör_cal(pcm, ornek, hz);
}
free(wav);
}

static void tur(void)
{
    durum_yaz(DURUM_BEKLIYOR);
    mik_bosalt();
    const char *nasil = tetik_bekle();
    ESP_LOGI(ETIKET, "(%s ile uyandı)", nasil);

    durum_yaz(DURUM_DINLIYOR);
    size_t wav_boy = 0;
    uint8_t *wav = sessizlige_kadar_kaydet(&wav_boy);
    if (!wav) return;

    /* Bundan sonrası mikrofonu gerektirmiyor. Kapatmak Tepecan'ın kendi
    * sesini duymasını da engelliyor. */
    mik_duraklat();
    durum_yaz(DURUM_DUSUNUYOR);

    char *soru = beyin_stt(wav, wav_boy);
    free(wav);
    if (!soru || soru[0] == '\0') {
        free(soru);
        mik_devam();
        return;
    }
    ESP_LOGI(ETIKET, "sen: %s", soru);

    char *cevap = beyin_chat(soru);
    free(soru);
    if (cevap) {
        ESP_LOGI(ETIKET, "tepecan: %s", cevap);
        konus(cevap);
        free(cevap);
    }
    mik_devam();
}

/* ----- giriş */
static void spiffs_bagla(void)
{
    esp_vfs_spiffs_conf_t c = {
        .base_path = "/model",
        .partition_label = "model",
        .max_files = 2,
        .format_if_mount_failed = true,
    };
    esp_err_t r = esp_vfs_spiffs_register(&c);
    if (r != ESP_OK) ESP_LOGW(ETIKET, "SPIFFS bağlanamadı: %s", esp_err_to_name(r));
}

void app_main(void)
{
    ESP_LOGI(ETIKET, "Tepecan başlıyor");
    buton_kur();
    spiffs_bagla();
    ESP_ERROR_CHECK(ses_baslat());

    esp_err_t r = ag_baslat();
    if (r == ESP_ERR_NOT_FINISHED) {
        /* Kurulum portalı açık; kullanıcı ağ bilgisini girince kart kendi
        * yeniden başlıyor. Burada beklemekten başka yapacak iş yok. */
        while (true) vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

```

```
if (r != ESP_OK) ESP_LOGW(ETIKET, "ağa bağlanılamadı, denemeye devam");

if (!uyandırma_baslat(WAKE_MODEL_YOLU))
    ESP_LOGW(ETIKET, "uyandırma kelimesi kapalı - yalnız buton tetikler");

ESP_LOGI(ETIKET, "hazır. beyin: %s", beyin_adresi());
while (true) tur();
}
```

yazilim/esp32/main/ses.c — I2S mikrofon ve hoparlör

```
#include "ses.h"
#include "tepecan.h"

#include <string.h>
#include <math.h>
#include "driver/i2s_std.h"
#include "esp_log.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

static const char *ETIKET = "ses";

static i2s_chan_handle_t mik_kanal;
static i2s_chan_handle_t hop_kanal;
static bool mik_acik;

/* INMP441 24 bitlik veriyi 32 bitlik yuvada gönderiyor; 32 bit okuyup
 * yukarı hizalanmış 16 biti alıyoruz. */
#define MIK_YUVA I2S_DATA_BIT_WIDTH_32BIT

static esp_err_t mikrofon_kur(void)
{
    i2s_chan_config_t kanal = I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_0, I2S_ROLE_MASTER);
    kanal.dma_desc_num = 6;
    kanal.dma_frame_num = BLOK_ORNEK;
    kanal.auto_clear = true;
    ESP_ERROR_CHECK(i2s_new_channel(&kanal, NULL, &mik_kanal));

    i2s_std_config_t std = {
        .clk_cfg = I2S_STD_CLK_DEFAULT_CONFIG(ORNEK_HZ),
        .slot_cfg = I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(
            MIK_YUVA, I2S_SLOT_MODE_MONO),
        .gpio_cfg = {
            .mclk = I2S_GPIO_UNUSED,
            .bclk = PIN_MIC_SCK,
            .ws = PIN_MIC_WS,
            .dout = I2S_GPIO_UNUSED,
            .din = PIN_MIC_SD,
            .invert_flags = { false, false, false },
        },
    };
    std.slot_cfg.slot_mask = MIC_KANAL_SOL ? I2S_STD_SLOT_LEFT : I2S_STD_SLOT_RIGHT;
    ESP_ERROR_CHECK(i2s_channel_init_std_mode(mik_kanal, &std));
    ESP_ERROR_CHECK(i2s_channel_enable(mik_kanal));
    mik_acik = true;
    return ESP_OK;
}

static esp_err_t hoparlör_kur(void)
{
    i2s_chan_config_t kanal = I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_1, I2S_ROLE_MASTER);
    kanal.dma_desc_num = 8;
    kanal.dma_frame_num = 240;
    kanal.auto_clear = true;
    ESP_ERROR_CHECK(i2s_new_channel(&kanal, &hop_kanal, NULL));

    i2s_std_config_t std = {
        .clk_cfg = I2S_STD_CLK_DEFAULT_CONFIG(ORNEK_HZ),
        .slot_cfg = I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(
            I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_MONO),
        .gpio_cfg = {
            .mclk = I2S_GPIO_UNUSED,
            .bclk = PIN_SPK_BCLK,
            .ws = PIN_SPK_LRC,
            .dout = PIN_SPK_DIN,
            .din = I2S_GPIO_UNUSED,
            .invert_flags = { false, false, false },
        },
    };
    ESP_ERROR_CHECK(i2s_channel_init_std_mode(hop_kanal, &std));
    return ESP_OK;
}

esp_err_t ses_baslat(void)
```

```

{
    ESP_ERROR_CHECK(mikrofon_kur());
    ESP_ERROR_CHECK(hoparlör_kur());
    ESP_LOGI(ETIKET, "I2S hazır: mik %d/%d/%d, hoparlör %d/%d/%d",
        PIN_MIC_SCK, PIN_MIC_WS, PIN_MIC_SD,
        PIN_SPK_BCLK, PIN_SPK_LRC, PIN_SPK_DIN);
    return ESP_OK;
}

esp_err_t mik_oku(int16_t *hedef, size_t ornek)
{
    static int32_t ham[BLOK_ORNEK];
    size_t okundu = 0;
    if (ornek > BLOK_ORNEK) ornek = BLOK_ORNEK;

    esp_err_t r = i2s_channel_read(mik_kanal, ham, ornek * sizeof(int32_t),
        &okundu, portMAX_DELAY);
    if (r != ESP_OK) return r;

    size_t n = okundu / sizeof(int32_t);
    for (size_t i = 0; i < n; i++) {
        /* INMP441: anlamlı 24 bit üstte. 16 bite indirmek için 11 kaydırıp
        * (8 bit hizalama + 3 bit kazanç) taşmaya karşı kırıyoruz. */
        int32_t v = ham[i] >> 11;
        if (v > 32767) v = 32767;
        if (v < -32768) v = -32768;
        hedef[i] = (int16_t)v;
    }
    for (size_t i = n; i < ornek; i++) hedef[i] = 0;
    return ESP_OK;
}

void mik_duraklat(void)
{
    if (mik_acik) { i2s_channel_disable(mik_kanal); mik_acik = false; }
}

void mik_devam(void)
{
    if (!mik_acik) { i2s_channel_enable(mik_kanal); mik_acik = true; }
}

void mik_bosalt(void)
{
    /* Tetikten önce biriken sesi at, yoksa kayıt eski gürültüyle başlıyor. */
    static int16_t cop[BLOK_ORNEK];
    for (int i = 0; i < 8; i++) mik_oku(cop, BLOK_ORNEK);
}

esp_err_t hoparlör_cal(const int16_t *pcm, size_t ornek, uint32_t hz)
{
    /* Sunucudan gelen WAV 22.05 kHz olabiliyor (Piper), mikrofon 16 kHz.
    * Kanalı çalışmadan önce o hıza ayarlıyoruz. */
    i2s_std_clk_config_t saat = I2S_STD_CLK_DEFAULT_CONFIG(hz);
    ESP_ERROR_CHECK(i2s_channel_disable(hop_kanal));
    ESP_ERROR_CHECK(i2s_channel_reconfig_std_clock(hop_kanal, &saat));
    ESP_ERROR_CHECK(i2s_channel_enable(hop_kanal));

    size_t yazildi = 0;
    esp_err_t r = i2s_channel_write(hop_kanal, pcm, ornek * sizeof(int16_t),
        &yazildi, portMAX_DELAY);
    /* Son tamponun gerçekten çıkması için kısa bir bekleme; hemen disable
    * edersek cümlelerin sonu kesiliyor. */
    vTaskDelay(pdMS_TO_TICKS(60));
    i2s_channel_disable(hop_kanal);
    return r;
}

bool ses_var_mi(const int16_t *blok, size_t ornek)
{
    int64_t kare = 0;
    for (size_t i = 0; i < ornek; i++) kare += (int32_t)blok[i] * blok[i];
    double rms = sqrt((double)kare / (double)ornek);
    return rms > 500.0; /* sessiz oda ~100-200, konuşma >1000 */
}

```

yazilim/esp32/main/ag.c — Wi-Fi ve beyin sunucusu istemcisi

```
#include "ag.h"
#include "tepecan.h"

#include <string.h>
#include <stdlib.h>
#include "esp_log.h"
#include "esp_wifi.h"
#include "esp_netif.h"
#include "esp_event.h"
#include "esp_http_client.h"
#include "esp_heap_caps.h"
#include "nvs_flash.h"
#include "nvs.h"
#include "cJSON.h"
#include "freertos/FreeRTOS.h"
#include "freertos/event_groups.h"

static const char *ETIKET = "ag";
static EventGroupHandle_t olaylar;
#define BIT_BAGLANDI BIT0
static char g_beyin[128] = BEYIN_VARSAYILAN;
static bool g_bagli;

/* ----- NVS */
static esp_err_t nvs_oku(const char *anahtar, char *hedef, size_t boy)
{
    nvs_handle_t h;
    if (nvs_open("tepecan", NVS_READONLY, &h) != ESP_OK) return ESP_FAIL;
    esp_err_t r = nvs_get_str(h, anahtar, hedef, &boy);
    nvs_close(h);
    return r;
}

esp_err_t nvs_yaz(const char *anahtar, const char *deger)
{
    nvs_handle_t h;
    if (nvs_open("tepecan", NVS_READWRITE, &h) != ESP_OK) return ESP_FAIL;
    esp_err_t r = nvs_set_str(h, anahtar, deger);
    if (r == ESP_OK) r = nvs_commit(h);
    nvs_close(h);
    return r;
}

/* ----- olaylar */
static void olay(void *arg, esp_event_base_t taban, int32_t id, void *veri)
{
    if (taban == WIFI_EVENT && id == WIFI_EVENT_STA_START) {
        esp_wifi_connect();
    } else if (taban == WIFI_EVENT && id == WIFI_EVENT_STA_DISCONNECTED) {
        g_bagli = false;
        xEventGroupClearBits(olaylar, BIT_BAGLANDI);
        ESP_LOGW(ETIKET, "bağlantı koptu, yeniden deneniyor");
        vTaskDelay(pdMS_TO_TICKS(2000));
        esp_wifi_connect();
    } else if (taban == IP_EVENT && id == IP_EVENT_STA_GOT_IP) {
        ip_event_got_ip_t *e = veri;
        ESP_LOGI(ETIKET, "IP: " IPSTR, IP2STR(&e->ip_info.ip));
        g_bagli = true;
        xEventGroupSetBits(olaylar, BIT_BAGLANDI);
    }
}

esp_err_t ag_baslat(void)
{
    esp_err_t r = nvs_flash_init();
    if (r == ESP_ERR_NVS_NO_FREE_PAGES || r == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ESP_ERROR_CHECK(nvs_flash_init());
    }
    nvs_oku("beyin", g_beyin, sizeof(g_beyin));

    char ssid[33] = {0}, sifre[65] = {0};
    if (nvs_oku("ssid", ssid, sizeof(ssid)) != ESP_OK || ssid[0] == '\0') {
        ESP_LOGW(ETIKET, "kayıtlı ağ yok - kurulum noktası açılıyor");
    }
}
```

```

        return kurulum_noktasi_ac();          /* portal.c */
    }
    nvs_oku("sifre", sifre, sizeof(sifre));

    olaylar = xEventGroupCreate();
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    esp_netif_create_default_wifi_sta();

    wifi_init_config_t ayar = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&ayar));
    ESP_ERROR_CHECK(esp_event_handler_instance_register(
        WIFI_EVENT, ESP_EVENT_ANY_ID, &olay, NULL, NULL));
    ESP_ERROR_CHECK(esp_event_handler_instance_register(
        IP_EVENT, IP_EVENT_STA_GOT_IP, &olay, NULL, NULL));

    wifi_config_t wc = { 0 };
    strncpy((char *)wc.sta.ssid, ssid, sizeof(wc.sta.ssid));
    strncpy((char *)wc.sta.password, sifre, sizeof(wc.sta.password));
    ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
    ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wc));
    ESP_ERROR_CHECK(esp_wifi_start());

    ESP_LOGI(ETIKET, "'%s' ağına bağlanılıyor, beyin: %s", ssid, g_beyin);
    xEventGroupWaitBits(olaylar, BIT_BAGLANDI, pdFALSE, pdTRUE,
        pdMS_TO_TICKS(30000));
    return g_bagli ? ESP_OK : ESP_ERR_TIMEOUT;
}

bool ag_bagli(void) { return g_bagli; }
const char *beyin_adresi(void) { return g_beyin; }

/* ----- HTTP yardım */
typedef struct { char *veri; size_t boy; size_t sinir; } tampon_t;

static esp_err_t topla(esp_http_client_event_t *e)
{
    tampon_t *t = e->user_data;
    if (e->event_id != HTTP_EVENT_ON_DATA || !t) return ESP_OK;
    if (t->boy + e->data_len + 1 > t->sinir) return ESP_OK; /* taşma koruması */
    memcpy(t->veri + t->boy, e->data, e->data_len);
    t->boy += e->data_len;
    t->veri[t->boy] = '\0';
    return ESP_OK;
}

static char *gonder(const char *yol, const char *tur,
    const void *govde, size_t govde_boy, size_t sinir,
    size_t *gelen_boy)
{
    char url[192];
    snprintf(url, sizeof(url), "%s%s", g_beyin, yol);

    char *cikti = heap_caps_malloc(sinir, MALLOC_CAP_SPIRAM);
    if (!cikti) { ESP_LOGE(ETIKET, "PSRAM yetmedi (%u)", (unsigned)sinir); return NULL; }
    cikti[0] = '\0';
    tampon_t t = { .veri = cikti, .boy = 0, .sinir = sinir };

    esp_http_client_config_t ayar = {
        .url = url,
        .method = HTTP_METHOD_POST,
        .timeout_ms = HTTP_ZAMAN_ASIMI_MS,
        .event_handler = topla,
        .user_data = &t,
        .buffer_size = 2048,
    };
    esp_http_client_handle_t c = esp_http_client_init(&ayar);
    esp_http_client_set_header(c, "Content-Type", tur);
    esp_http_client_set_post_field(c, (const char *)govde, govde_boy);

    esp_err_t r = esp_http_client_perform(c);
    int kod = esp_http_client_get_status_code(c);
    esp_http_client_cleanup(c);

    if (r != ESP_OK || kod != 200) {
        ESP_LOGE(ETIKET, "%s başarısız (%s, HTTP %d)", yol, esp_err_to_name(r), kod);
    }
}

```

```

        free(cikti);
        return NULL;
    }
    t.veri[t.boy] = '\0';
    if (gelen_boy) *gelen_boy = t.boy;
    return cikti;
}

/* JSON'dan tek bir metin alanı çek. */
static char *alan(char *json, const char *ad)
{
    if (!json) return NULL;
    cJSON *k = cJSON_Parse(json);
    free(json);
    if (!k) return NULL;
    cJSON *d = cJSON_GetObjectItem(k, ad);
    char *sonuc = (cJSON_IsString(d) && d->valuelstring) ? strdup(d->valuelstring) : NULL;
    cJSON_Delete(k);
    return sonuc;
}

char *beyin_stt(const uint8_t *wav, size_t uzunluk)
{
    /* /stt-raw: ham WAV gövdesi. Çok parçalı form kurmak ESP32'de gereksiz
     * zahmet, sunucuya bunun için ayrı bir uç eklendi. */
    return alan(gonder("/stt-raw", "audio/wav", wav, uzunluk, 4096, NULL), "text");
}

char *beyin_chat(const char *soru)
{
    cJSON *k = cJSON_CreateObject();
    cJSON_AddStringToObject(k, "text", soru);
    char *govde = cJSON_PrintUnformatted(k);
    cJSON_Delete(k);

    char *yanit = gonder("/chat", "application/json", govde, strlen(govde), 8192, NULL);
    free(govde);
    return alan(yanit, "text");
}

uint8_t *beyin_tts(const char *metin, size_t *uzunluk)
{
    cJSON *k = cJSON_CreateObject();
    cJSON_AddStringToObject(k, "text", metin);
    char *govde = cJSON_PrintUnformatted(k);
    cJSON_Delete(k);

    /* Piper WAV'ı uzun cümlelerde birkaç yüz KB olabiliyor; PSRAM'de duruyor. */
    const size_t sinir = 1024 * 1024;
    size_t gelen = 0;
    char *wav = gonder("/tts", "application/json", govde, strlen(govde), sinir, &gelen);
    free(govde);
    if (!wav || gelen < 44) { free(wav); *uzunluk = 0; return NULL; }

    /* Gerçekten gelen bayt sayısını kullanıyoruz. Başlıktaki RIFF uzunluğuna
     * güvenmek, yanıt kırıldığında tamponun dışını okumak demek olurdu. */
    *uzunluk = gelen;
    return (uint8_t *)wav;
}

```

yazilim/esp32/main/portal.c — ilk kurulum sayfası

```
/* İlk kurulum: kart kendi erişim noktasını açıyor, telefonda ağ ve beyin
 * adresi giriliyor, NVS'e yazılıp yeniden başlıyor.
 *
 * Böylece SSID'yi koda gömmek gerekmiyor: maskot kulüpten etkinliğe
 * giderken ağ değiştirmek için bilgisayara bağlanmaya gerek kalmıyor.
 */
#include <string.h>
#include <stdlib.h>
#include "esp_log.h"
#include "esp_wifi.h"
#include "esp_netif.h"
#include "esp_event.h"
#include "esp_http_server.h"
#include "esp_system.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

#include "ag.h"

static const char *ETIKET = "portal";

#define AP_SSID "Tepecan-Kurulum"

static const char SAYFA[] =
    "<!doctype html><meta charset=utf-8>"
    "<meta name=viewport content='width=device-width,initial-scale=1'>"
    "<title>Tepecan kurulum</title>"
    "<style>body{font-family:system-ui;margin:0;padding:24px;background:#12161c;"
    "color:#e8eef6}h1{font-size:20px;color:#5aa9f8}label{display:block;margin:14px 0 4px}"
    "input{width:100%;box-sizing:border-box;padding:10px;border-radius:8px;"
    "border:1px solid #33404f;background:#1b222c;color:#e8eef6;font-size:16px}"
    "button{margin-top:20px;width:100%;padding:12px;border:0;border-radius:8px;"
    "background:#1f5fa9;color:#fff;font-size:16px}</style>"
    "<h1>Tepecan kurulum</h1>"
    "<form method=post action=/kaydet>"
    "<label>Wi-Fi adi</label><input name=ssid required>"
    "<label>Wi-Fi sifresi</label><input name=sifre type=password>"
    "<label>Beyin sunucusu</label>"
    "<input name=beyin value='http://tepecan-beyin.local:8000'>"
    "<button>Kaydet ve yeniden baslat</button></form>";

static esp_err_t sayfa_ver(httpd_req_t *r)
{
    httpd_resp_set_type(r, "text/html; charset=utf-8");
    return httpd_resp_send(r, SAYFA, HTTPD_RESP_USE_STRLEN);
}

/* application/x-www-form-urlencoded çözücü: yalnız bize lazım olan kadarı. */
static void alan_cek(const char *govde, const char *ad, char *hedef, size_t boy)
{
    hedef[0] = '\0';
    char desen[24];
    snprintf(desen, sizeof(desen), "%s=", ad);
    const char *p = strstr(govde, desen);
    if (!p) return;
    p += strlen(desen);

    size_t i = 0;
    while (*p && *p != '&' && i + 1 < boy) {
        if (*p == '+') { hedef[i++] = ' '; p++; }
        else if (*p == '%' && p[1] && p[2]) {
            char h[3] = { p[1], p[2], 0 };
            hedef[i++] = (char)strtol(h, NULL, 16);
            p += 3;
        } else hedef[i++] = *p++;
    }
    hedef[i] = '\0';
}

static esp_err_t kaydet(httpd_req_t *r)
{
    char govde[512] = { 0 };
    int n = httpd_req_recv(r, govde, sizeof(govde) - 1);
    if (n <= 0) return ESP_FAIL;
```

```

govde[n] = '\0';

char ssid[64], sifre[96], beyin[160];
alan_cek(govde, "ssid", ssid, sizeof(ssid));
alan_cek(govde, "sifre", sifre, sizeof(sifre));
alan_cek(govde, "beyin", beyin, sizeof(beyin));

if (ssid[0] == '\0') {
    httpd_resp_send(r, "Wi-Fi adi bos olamaz.", HTTPD_RESP_USE_STRLEN);
    return ESP_OK;
}
nvs_yaz("ssid", ssid);
nvs_yaz("sifre", sifre);
if (beyin[0]) nvs_yaz("beyin", beyin);

httpd_resp_set_type(r, "text/html; charset=utf-8");
httpd_resp_send(r, "<meta charset=utf-8><h2>Kaydedildi. Yeniden baslatiliyor...</h2>",
    HTTPD_RESP_USE_STRLEN);
ESP_LOGI(ETIKET, "'%s' kaydedildi, yeniden baslatiliyor", ssid);
vTaskDelay(pdMS_TO_TICKS(800));
esp_restart();
return ESP_OK;
}

esp_err_t kurulum_noktasi_ac(void)
{
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    esp_netif_create_default_wifi_ap();

    wifi_init_config_t ayar = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&ayar));

    wifi_config_t wc = { 0 };
    strncpy((char *)wc.ap.ssid, AP_SSID, sizeof(wc.ap.ssid));
    wc.ap.ssid_len = strlen(AP_SSID);
    wc.ap.max_connection = 3;
    wc.ap.authmode = WIFI_AUTH_OPEN; /* şifresiz: kurulumu kolaylaştırıyor */
    ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_AP));
    ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_AP, &wc));
    ESP_ERROR_CHECK(esp_wifi_start());

    httpd_handle_t sunucu = NULL;
    httpd_config_t hc = HTTPD_DEFAULT_CONFIG();
    ESP_ERROR_CHECK(httpd_start(&sunucu, &hc));

    httpd_uri_t kok = { .uri = "/", .method = HTTP_GET, .handler = sayfa_ver };
    httpd_uri_t post = { .uri = "/kaydet", .method = HTTP_POST, .handler = kaydet };
    httpd_register_uri_handler(sunucu, &kok);
    httpd_register_uri_handler(sunucu, &post);

    ESP_LOGW(ETIKET, "'%s' ağına bağlanıp http://192.168.4.1 adresini aç", AP_SSID);
    return ESP_ERR_NOT_FINISHED; /* main bunu görünce beklemeye geçiyor */
}

```

yazilim/sunucu/persona.txt — kişilik

Sen Tepecan'sın: IEEE Yeditepe Üniversitesi Öğrenci Kolu'nun maskotu, mavi metalik bir robotsun. Kulübün standında insanlarla sohbet ediyorsun.

Nasıl konuşursun:

- Türkçe, samimi ve sıcak. Karşındaki üniversite öğrencisi.
- KISA konuş. En fazla 2-3 cümle. Sesli konuştuğun için uzun cevap sıkıcı olur.
- Neşeli ve meraklısın ama abartılı değilsin. Ara sıra robot esprisi yaparsın.
- Madde işareti, emoji, markdown kullanma. Düz konuşma metni yaz.
- Rakamları ve kısaltmaları okunacak şekilde yaz (IEEE yerine "aye triple i").

Kurallar:

- Bilmediğin bir şey sorulursa uydurma. "Onu bilmiyorum ama standdaki arkadaşlara sorabilirsin" de.
- Kişisel veri isteme, kaydetme.
- Siyaset ve tartışmalı konulara girme, nazikçe konuyu değiştir.
- Sana kim olduğun sorulursa: kulübün maskotusun, içinde küçük bir bilgisayar var.

yazilim/sunucu/bilgiler.txt — doldurulacak bilgi dosyası

Tepecan'ın bildikleri. Buraya kulübe dair gerçekleri yaz - küçük modelin
doğru cevap vermesini sağlayan asıl şey burası. Kısa ve net cümleler kullan.
1500 karakteri geçmemeye çalış, yoksa küçük modelin dikkati dağılır.

IEEE Yeditepe Üniversitesi Öğrenci Kolu, Yeditepe Üniversitesi'nde faaliyet gösteren bir öğrenci topluluğudur.

Kulüpte ... (alt takımları buraya yaz)
Etkinlikler: ... (düzenli etkinlikleri buraya yaz)
Başvuru: ... (nasıl üye olunur)
Sosyal medya: ... (hesaplar)

Tepecan hakkında: 3D yazıcıda basıldı, 24 santimetre boyunda. İçinde bir Raspberry Pi var. Sırtındaki kapak vidayla açılıyor.

yazilim/sunucu/requirements.txt

```
faster-whisper
fastapi
uvicorn[standard]
python-multipart
pydantic
requests
```